

10 — Script de démonstration détaillé (≈15 min, avec répliques mot à mot)

Légende : 🖱️ = action à l'écran · 🗣️ = **texte à réciter**. Support : <https://upcycleconnect.pro/> (prod) · onglet local <http://localhost:8088> en secours. **Carte bancaire de test Stripe :** 4242 4242 4242 4242 , date future quelconque (ex 12/34), CVC quelconque (ex 123). **Préparer avant :** avoir en base **un événement payant** validé (pour l'inscription), et **une réservation de casier active** avec son **PIN** et son **code-barres** (pour le simulateur). Noter ces codes sur un papier.

Minutage

Séquence	Fin visée
0. Intro + connexion	1:15
1. Particulier (annonce)	3:15
2. Box / conteneurs (simulateur)	5:00
3. Salarié (créer un événement)	6:45
4. Paiement + inscription (Stripe)	9:00
5. Professionnel	10:30
6. Administrateur	13:30
7. Technique + conclusion	15:00

SÉQUENCE 0 — Introduction (0:00 → 1:15)

🖱️ Page d'accueil <https://upcycleconnect.pro/>.

🗣️ « Bonjour. Je vous présente **UpcycleConnect**, notre plateforme d'économie circulaire développée pour **Renova Systems**. »

🗣️ « Le principe : les **particuliers** donnent ou vendent des objets, les **professionnels** récupèrent la matière, et tout transite par un **réseau de conteneurs connectés**. Le site est **déployé en ligne**, en HTTPS, sur une infrastructure Docker. »

🗣️ « L'application gère **quatre rôles**, chacun avec son tableau de bord. Je vais tous vous les montrer, ainsi que le **paiement** et le **parcours des conteneurs**. Commençons par le particulier. »

🖱️ Connexion → compte particulier → `/client`.

SÉQUENCE 1 — Particulier : l'annonce (1:15 → 3:15)

🗣️ « Voici l'espace particulier. En haut, trois indicateurs **calculés en temps réel** : annonces, dépôts actifs, et score écologique. »

🖱️ Cliquer « + Ajouter une annonce », remplir titre, catégorie, type Vente, prix, description, ajouter une photo, puis Soumettre.

👤 « La fonction principale : publier une annonce. Je choisis une **catégorie** — elles viennent de la base, gérées par l'admin — j'ajoute une **photo**, et je soumetts. »

👤 « Point clé : l'annonce n'est **pas publiée directement**, elle passe en **validation**. On la retrouvera dans le back-office admin. »

📁 Aller sur `/annonces`.

👤 « Voici la marketplace, avec **filtres par catégorie** et recherche. Notez la couleur **violette** de l'interface particulier — elle changera pour le professionnel. »

SÉQUENCE 2 — Les conteneurs & le simulateur (3:15 → 5:00)

👤 « Le cœur du concept, c'est le **réseau de conteneurs**. Quand un objet est vendu, le système réserve un **casier** et génère deux codes : un **code PIN** pour que le vendeur dépose l'objet, et un **code-barres** pour que l'acheteur le récupère. »

📁 Ouvrir `/simulateur`.

👤 « Comme nous n'avons pas de serrure physique ici, voici un **simulateur** qui reproduit le boîtier électronique du conteneur. »

📁 Dans « Digicode Vendeur » (`#pinDepotInput`), saisir le PIN noté → valider.

👤 « Côté vendeur, je saisis le **code PIN**... et le système confirme le **dépôt** : le casier passe en "occupé", et l'objet est marqué comme déposé. »

📁 Dans « Digicode Acheteur » (`#pinRetraitInput`), saisir le code-barres (ex `UC-7-1`) → valider.

👤 « Côté acheteur, je scanne le **code-barres**... et là, le retrait est validé : le casier se **libère**, l'objet passe en "récupéré", et — c'est important — l'utilisateur gagne des points sur son **Upcycling Score**. »

👤 « Tout le cycle — réservation, dépôt, retrait — est tracé en base dans une table dédiée. Passons à la création de contenu par l'équipe interne. »

SÉQUENCE 3 — Salarié : créer un événement (5:00 → 6:45)

📁 Se reconnecter avec le compte salarié → `/salarie` → `/salarie/evenements`.

👤 « Voici l'espace salarié, l'équipe interne. Sa fonction clé : créer des **événements et formations**. »

📁 Ouvrir le formulaire, montrer les champs (titre, date, lieu, tarif, image, PDF).

👤 « Je renseigne un titre, une date, un lieu, un **tarif** — car un événement peut être payant — une image, et je peux joindre un **PDF** de ressources. »

👤 « Deux règles métier importantes : l'événement part **en attente de validation** de l'admin, et pour proposer un événement **payant**, le salarié doit avoir un **compte Stripe** — vérifié **côté serveur**. »

SÉQUENCE 4 — Paiement & inscription à un événement (6:45 → 9:00) ★

📁 Se reconnecter avec le compte particulier (ou pro) → aller sur `/evenements`.

👤 « Maintenant, la partie **paiement**, essentielle sur une plateforme comme celle-ci. Un utilisateur veut s'inscrire à une **formation payante**. »

📁 Sur la carte d'un événement payant, cliquer « S'inscrire → ».

🗣️ « Je clique sur **S'inscrire**. Comme l'événement est payant, l'application appelle notre API, qui crée une **session de paiement Stripe** et me redirige vers la page de paiement sécurisée. »

📁 La page Stripe Checkout s'affiche. Saisir la carte de test `4242 4242 4242 4242`, date `12/34`, CVC `123`, puis payer.

🗣️ « Nous sommes maintenant sur **Stripe**, en mode test. Je saisis une carte de test... et je valide le paiement. »

🗣️ « Techniquement, le paiement passe par **Stripe Connect** : l'argent va au salarié organisateur, **moins une commission de 5 %** qui revient à la plateforme. Une **facture PDF** est générée automatiquement, et l'inscription est confirmée par un **webhook** Stripe. »

⚠️ Si le retour de paiement échoue en live : rester sur la page Stripe (c'est déjà la preuve de l'intégration réelle) et enchaîner. Alternative sûre : s'inscrire à un événement gratuit pour montrer l'inscription instantanée.

SÉQUENCE 5 — Professionnel (9:00 → 10:30)

📁 Se reconnecter avec le compte pro → `/pro`.

🗣️ « Voici l'espace professionnel. Première chose : l'interface est en **teal**, plus en violet — l'application **s'adapte au rôle**, y compris sur la page des annonces. »

📁 Pointer l'encart abonnement et un projet avant/après.

🗣️ « Le pro fonctionne en **Freemium / Premium** : l'abonnement Premium, lui aussi payé via Stripe, débloque des tableaux avancés et une meilleure visibilité. Il documente ses **projets d'upcycling** avec des photos **avant / après** et le **CO₂ évité**, et peut **sponsoriser** ses annonces. »

SÉQUENCE 6 — Administrateur (10:30 → 13:30) — POINT FORT

📁 Se reconnecter avec le compte admin → `/admin`.

🗣️ « Le cœur de la plateforme : le back-office. La Vue d'ensemble affiche les KPI — utilisateurs, **revenus du mois issus des commissions**, conteneurs — l'activité récente, et un badge des éléments à valider. »

📁 `/admin/validations` → approuver l'annonce créée au début.

🗣️ « Et voici l'annonce que j'ai créée tout à l'heure. Je l'**approuve**... et elle **disparaît en direct** de la liste : elle est maintenant publiée. »

📁 `/admin/users`.

🗣️ « La gestion des utilisateurs : liste **paginée**, recherche, et je peux **valider, refuser ou bannir** un compte — par exemple contrôler un professionnel avant de l'activer. »

📁 `/admin/conteneurs` → ouvrir un conteneur → « 📄 Rapport logistique » (PDF se télécharge).

🗣️ « La logistique : je gère les conteneurs et l'état des casiers, et je génère un **rapport PDF**... voilà, il est téléchargé. »

📁 `/admin/finances`.

🗣️ « Les finances : le volume d'affaires et les **revenus de la commission de 5 %**, avec le détail des transactions Stripe. »

🖱 Revenir sur `/admin`, pointer *Catégories / Langues / Notification*.

🗣 « Enfin, l'admin gère les **catégories**, l'ajout d'une **langue** par import de fichier, et l'**envoi de notifications**. Et tout ceci est protégé **côté serveur** par un contrôle de rôle : un particulier ne peut jamais y accéder. »

SÉQUENCE 7 — Technique & conclusion (13:30 → 15:00)

🖱 Terminal : `docker ps`.

🗣 « Techniquement : l'application tourne en **trois conteneurs Docker** — MySQL, l'API Go, et Nginx. Front et back sont **séparés** et orchestrés par Docker Compose. »

🖱 Postman/curl : `login` → route protégée.

🗣 « L'API est en **Go**, sécurisée par **JWT** : sans jeton valide, elle répond 401. La sécurité est côté serveur. »

🖱 Navigateur : pointer le cadenas **HTTPS**.

🗣 « Et tout est **réellement en ligne** : `upcycleconnect.pro`, en HTTPS, derrière Nginx, sur IP publique. »

🗣 « **Pour résumer** : quatre rôles et quatre tableaux de bord, le parcours complet d'un objet du dépôt à la récupération via nos conteneurs, un **paiement sécurisé par Stripe avec commission**, une API Go protégée par JWT, le tout **déployé en production** avec Docker, Nginx et HTTPS. Merci, je suis prêt pour vos questions. »

À ÉVITER / précautions

- Le **retour** après paiement Stripe peut échouer en live → rester sur la page Stripe (preuve suffisante) ou utiliser un événement **gratuit** en secours.
- **Push OneSignal** : ne pas tester en local (HTTPS requis).
- Ne pas montrer de données brutes incohérentes (comptes « Rejeté », annonces `azerty...`).
- Bug live → passer à la **capture de secours** et continuer.

Plan B

- Prod HS → basculer sur `http://localhost:8088` (même parcours ; le paiement Stripe fonctionne aussi en test).
- Tout HS → dérouler avec les **captures d'écran** de secours (dont la page **Stripe Checkout** et un **PDF de facture**).

Check-list de préparation (à faire la veille)

- [] Un **événement payant** validé et visible sur `/evenements`.
- [] Une **réservation de casier active** → noter le **PIN** (dépôt) et le **code-barres** (retrait) pour le simulateur.
- [] Le salarié organisateur a bien un **stripe_account_id**.
- [] Carte de test Stripe notée : `4242 4242 4242 4242`.
- [] Captures de secours prêtes (Stripe Checkout, facture PDF, chaque dashboard).